

Proyecto Centinela — Fase 3: Pipeline, GPU y Despliegue

Sistema de alerta temprana del poder adquisitivo del SMLV

Estefanny Ruíz González^a
estefannyruiz@usantotomas.edu.co

Miguel Alarcón Ojeda^b
miguel.alarcon@usantotomas.edu.co

1. Propósito y adaptación al proyecto

La Fase 3 industrializa el sistema multimodal construido en la Fase 2 del Proyecto Centinela, asumiendo el rol de ingeniería MLOps. El escenario de referencia del enunciado (clasificación de hojas de café para AGROSAVIA) se adapta al problema propio del proyecto: predicción del régimen cambiario de la TRM y alerta temprana del poder adquisitivo del SMLV.

Rol en el enunciado	Nuestra implementación
Rama visual (CNN)	ResNet18 sobre matrices de recurrencia TRM
Rama temporal (RNN/LSTM/GRU)	GRU sobre retornos diarios de la TRM
Despliegue offline en tablet	Rama A cuantizada INT8, exportada a LiteRT
Despliegue en la nube	Rama B exportada a ONNX, servida por API REST
Fuente de datos (no Kaggle)	trm_diaria_limpia.csv — Superfinanciera Colombia

Tabla 1: Adaptación del escenario de referencia al Proyecto Centinela.

GPU utilizada: Tesla T4 (Google Colab), 15.6 GB VRAM, CUDA 13.0, Driver 580.82.07. PyTorch 2.11.0+cu128, TensorFlow 2.20.0.

2. Pipeline de datos — comparativa tf.data vs. DataLoader

Las matrices de recurrencia (6.277 imágenes PNG de 30×30 píxeles) se materializan en disco con estructura de carpetas por clase (`train/val/test/<clase>/`), permitiendo comparar los dos frameworks en igualdad de condiciones.

Principio del prefetch:

$$t_{\text{época}} \approx \text{máx}(t_{\text{CPU}}, t_{\text{GPU}})$$

Sin prefetch, la GPU espera a la CPU entre batches. Con prefetch, ambas trabajan en paralelo y el tiempo total se acerca al máximo de las dos, no a su suma (TensorFlow Authors, 2023).

Configuración PyTorch: `DataLoader(num.workers=2, pin_memory=True, prefetch_factor=2)`.

Configuración tf.data: `.map(AUTOTUNE).cache().shuffle(buffer).batch(32).prefetch(AUTOTUNE)`.

Data augmentation (≥ 4 transformaciones, solo en train):

^aEstudiante

^bEstudiante

Framework	Pasada 1 (s)	Pasada 2 (s)
PyTorch DataLoader	8.34	7.16
tf.data	8.42	0.67

Tabla 2: Tiempo de carga por época (20 batches, batch_size=32). tf.data baja fuertemente en la pasada 2 gracias a `.cache()`; DataLoader es más estable entre pasadas porque `num_workers` ya paraleliza desde la pasada 1.

Transformación	Justificación
RandomRotation($\pm 15^\circ$)	La orientación de los patrones de recurrencia no es intrínsecamente informativa.
RandomHorizontalFlip	Las matrices de recurrencia son simétricas respecto a la diagonal por construcción.
ColorJitter(bright=0.2)	Simula variaciones de intensidad sin alterar la estructura temporal.
RandomResizedCrop(0.85–1.0)	Simula incertidumbre en el borde de la ventana sin perder la diagonal principal.

Tabla 3: Transformaciones de data augmentation y su justificación. No se usa **RandomErasing** ni recortes agresivos porque podrían borrar la diagonal principal, la característica más diagnóstica de toda matriz de recurrencia.

3. Entrenamiento en GPU con precisión mixta

Se entrena ResNet18 preentrenado (Feature Extraction, capa final `fc 512→3`) con dos configuraciones para cuantificar el beneficio de la precisión mixta (Micikevicius et al., 2018):

- **FP32**: entrenamiento estándar sin optimizaciones de memoria.
- **AMP (FP16)**: `torch.cuda.amp.autocast + GradScaler` para evitar underflow numérico. En la T4 se usa `float16` (no `bfloat16`, que requiere GPU Ampere o superior).

Matiz importante: el ahorro de VRAM de AMP se da principalmente sobre las **activaciones** (FP16 $\approx$ mitad de espacio). El ahorro total es menor porque el optimizador Adam mantiene una copia de los pesos maestros en FP32 para las actualizaciones de gradiente.

Checkpointing: se guarda un checkpoint cada 5 épocas con `{state_dict, optim, epoca, loss_val}` para permitir reanudación ante cortes de cuota de cómputo.

Manejo de CUDA OOM: se forzó deliberadamente un `batch_size=8192` para provocar el error, se capturó con `torch.cuda.OutOfMemoryError`, se liberó la caché con `torch.cuda.empty_cache()` y se reintentó con `batch_size=32` — recuperación exitosa documentada en el notebook.

4. Optimización Edge y exportación multi-formato

4.1. Cuantización post-entrenamiento INT8

Para despliegue offline en tablet se aplica cuantización INT8 vía `tf.lite.TFLiteConverter` con `representative_dataset` de 100 imágenes reales de entrenamiento (requisito para calibrar los rangos de activación con datos reales, no aleatorios).

Configuración	Accuracy val (10 épocas)	Diferencia
Con augmentation	0.497	+0.027
Sin augmentation	0.470	ref.

Tabla 4: Efecto del data augmentation sobre la accuracy de validación (10 épocas, mismo modelo ResNet18, Feature Extraction).

Configuración	Tiempo (min)	Pico VRAM (MB)	Mejor val loss
FP32	4.31	347	1.0824
AMP (FP16)	4.21	299	1.0824
Ahorro VRAM	—	13.9 %	—
Aceleración	1.02×	—	—

Tabla 5: Comparativa FP32 vs. precisión mixta AMP (FP16), 20 épocas, checkpointing cada 5 épocas, GPU Tesla T4.

Discusión del trade-off: la cuantización INT8 reduce el tamaño del modelo en un 69.4 % (de 16.04 MB a 4.90 MB) con una caída de latencia de solo 0.81 ms/imagen (20.36 ms vs. 21.17 ms en float32) y, sorprendentemente, una **mejora** de accuracy (+0.003) y F1 macro (+0.036). Esta mejora es estadísticamente marginal y se explica por la varianza del conjunto de test (n=942), pero confirma que la cuantización INT8 no degrada el desempeño del modelo en este caso. La reducción de tamaño es el beneficio principal para despliegue en tablet de gama media sin GPU.

Verificación de fidelidad: se compara el modelo `.tflite` contra las predicciones de PyTorch en las mismas imágenes para distinguir pérdidas propias del modelo de bugs de conversión. La arquitectura Keras equivalente (EfficientNet-B0) fue validada durante el entrenamiento de la Fase 2, y la exportación ONNX se verificó con `np.allclose(ato1=1e-4)` entre PyTorch y ONNX Runtime — diferencia máxima registrada: $<1e-4$.

4.2. Exportación multi-formato y verificación

Formato	Archivo	Uso
PyTorch (.pth)	<code>centinela_fase3_resnet18.pth</code>	Re-entrenamiento, producción Python
ONNX	<code>centinela_fase3_resnet18.onnx</code>	Portabilidad, puente hacia LiteRT
Keras/SavedModel	<code>centinela_fase3_resnet18_tf/</code>	TF Serving, conversión a TFLite
LiteRT (float32)	<code>centinela_fase3_resnet18_float.tflite</code>	Tablet (sin cuantizar)
LiteRT (INT8)	<code>centinela_fase3_resnet18_int8.tflite</code>	Tablet offline (producción)
ONNX (GRU)	<code>centinela_fase3_gru.onnx</code>	API REST en la nube

Formatos de exportación generados. La verificación numérica PyTorch vs. ONNX Runtime sobre un batch real de test dio `np.allclose(ato1=1e-4)`: verificado — diferencia máxima $< 10^{-4}$.

Nota importante: `.pth` y `.keras` son dos implementaciones equivalentes, no el mismo objeto serializado en dos archivos. ONNX actúa como puente portable entre ambos frameworks (He et al., 2016).

Métrica	Antes (float32)	Después (INT8)	Cambio
Tamaño modelo (MB)	16.04	4.90	−69.4 %
Latencia CPU (ms/img)	21.17	20.36	1.04×
Accuracy (test)	0.335	0.338	+0.003 pp
F1 macro (test)	0.167	0.203	+0.036 pp

Tabla 6: Tabla de optimización Edge — Rama A visual (ResNet18). La latencia se mide en CPU con 1 hilo (`num.threads=1`) porque el beneficio de INT8 es una historia de despliegue en tablet sin GPU, no en la T4.

5. Ruta de despliegue y justificación de ingeniería

	Rama A (visual)	Rama B (temporal)
Modelo	ResNet18, Feature Extraction	GRU (2 capas, 64 unidades)
Formato	.tflite INT8	.onnx
Dónde corre	Offline, en tablet	Servidor en la nube
Runtime	LiteRT	ONNX Runtime + FastAPI
Por qué	Sin conectividad requerida	LSTM/GRU a LiteRT: poco fiable

Tabla 7: Arquitectura de despliegue dual. La rama temporal se sirve por API REST porque la exportación de capas recurrentes a LiteRT es notoriamente inestable (operadores no siempre soportados según la versión). Los datos de TRM que consume el GRU requieren conectividad de todas formas para actualizarse, así que depender de una API en la nube no añade una restricción nueva al escenario de campo.

Principios de IA responsable:

- **Transparencia:** el sistema muestra la confianza de la predicción junto a la clase — un usuario de campo necesita saber cuándo el modelo está inseguro antes de actuar.
- **Beneficio social:** el sistema anticipa presión sobre el poder adquisitivo del SMLV para que hogares y pequeños negocios tomen decisiones informadas, sin sustituir asesoría financiera formal.
- **Mitigación de falsos negativos** (no alertar una depreciación real): el umbral de alerta se calibra conservadoramente — es preferible una alerta revisable que una omitida.
- **Mitigación de falsos positivos** (alertas sin fundamento): se muestra siempre la confianza y se evitan notificaciones automáticas sin revisión humana para decisiones de alto impacto.

Referencias

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. En *Proceedings of CVPR* (pp. 770–778). <https://doi.org/10.1109/CVPR.2016.90>
- Mickevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., ... Wu, H. (2018). Mixed precision training. *ICLR 2018*. <https://arxiv.org/abs/1710.03740>
- PyTorch Foundation. (2024). *Data loading and processing tutorial*. https://pytorch.org/tutorials/beginner/data_loading_tutorial.html
- TensorFlow Authors. (2023). *Better performance with the tf.data API*. https://www.tensorflow.org/guide/data_performance
- Google AI Edge. (2024). *LiteRT overview*. <https://ai.google.dev/edge/litert>
- Superintendencia Financiera de Colombia. (2026). *Tasa de cambio representativa del mercado (TRM) — serie histórica diaria* [Conjunto de datos]. Datos Abiertos Colombia. <https://datos.gov.co/Econom-a-y-Finanzas/TRM/ceyp-9c7c>